

WS73V100 Linux 平台驱动移植

用户指南

文档版本 01

发布日期 2023-12-11

前言

概述

本文档介绍了 WS73V100 SDK 开发环境及 Linux 平台的驱动移植指导，用于帮助用户在快速了解开发环境后进行二次开发。

读者对象

本文档主要适用于以下工程师：

- 软件开发工程师
- 软件测试工程师
- 技术支持工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。

符号	说明
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
01	2023-12-11	第一次正式版本发布。 更新 “3.3 驱动加载步骤” 小节内容。
00B03	2023-12-01	- 更新 “1.2 Python 环境安装” 小节内容。 - 更新 “2.1 SDK 目录结构介绍” 中目录结构。 - 删除 “Menuconfig 配置” 小节内容。 - 更新 “2.2 编译 SDK” 中编译说明及注意事项。 - 删除 “内核 PATCH 修改” 小节内容。 - 新增 “3.3 驱动加载步骤” 小节内容。 - 更新 “3.4 Linux 平台 WiFi 移植” 中驱动与工具的移植步骤。 - 更新 “3.5 Linux 平台蓝牙移植” 中驱动移植步骤。 - 更新 “3.6 Linux 平台星闪移植” 中驱动移植步骤及工具获取方法。
00B02	2023-10-31	- 更新 “2.2.1 config 配置方法” 的驱动编译参数。 - 更新 “Menuconfig 配置” 的 Menuconfig

文档版本	发布日期	修改说明
		配置项说明。 • 新增 “3.2 单板配置文件配置” 小节内容。 • 更新 “3.4.3 WiFi 业务调试” 小节内容。 • 更新 “3.5.3 蓝牙业务调试” 小节内容。
00B01	2023-08-01	第一次临时版本发布。

目 录

前言i

1 开发环境搭建1

1.1 SDK 开发环境简介.....1

1.2 Python 环境安装.....2

2 SDK 环境搭建.....4

2.1 SDK 目录结构介绍.....4

2.2 编译 SDK5

2.2.1 config 配置方法.....5

2.2.2 驱动代码编译.....8

2.2.3 注意事项8

3 Linux 移植说明.....9

3.1 内核参数配置.....9

3.1.1 配置 CFG802119

3.1.2 配置 SDIO.....10

3.1.3 配置 USB11

3.1.4 配置 Netlink.....12

3.1.5 配置 NAT 转发（可选）13

3.2 单板配置文件配置.....13

3.3 驱动加载步骤.....13

3.4 Linux 平台 WiFi 移植.....14

3.4.1 驱动移植步骤.....14

3.4.2 工具移植步骤.....15

3.4.3 WiFi 业务调试.....16

3.5 Linux 平台蓝牙移植 16

3.5.1 驱动移植步骤 16

3.5.2 工具移植步骤 17

3.5.3 蓝牙业务调试 22

3.6 Linux 平台星闪移植 22

3.6.1 驱动移植步骤 22

3.6.2 星闪工具获取 23

3.6.3 星闪业务调试 23

3.6.3.1 星闪设备检测 23

3.6.3.2 加载驱动 24

3.6.3.3 载入工具 24

3.6.3.4 星闪基础业务操作示例 25

4 常见问题 27

4.1 驱动加载出错 27

4.2 wpa_supplicant 启动失败 28

1 开发环境搭建

1.1 SDK 开发环境简介

1.2 Python 环境安装

1.1 SDK 开发环境简介

典型的 SDK 开发环境主要包括：

- Linux 服务器

Linux 服务器主要用于建立交叉编译环境，实现在 Linux 服务器上编译出可以在目标板上运行的可执行代码。

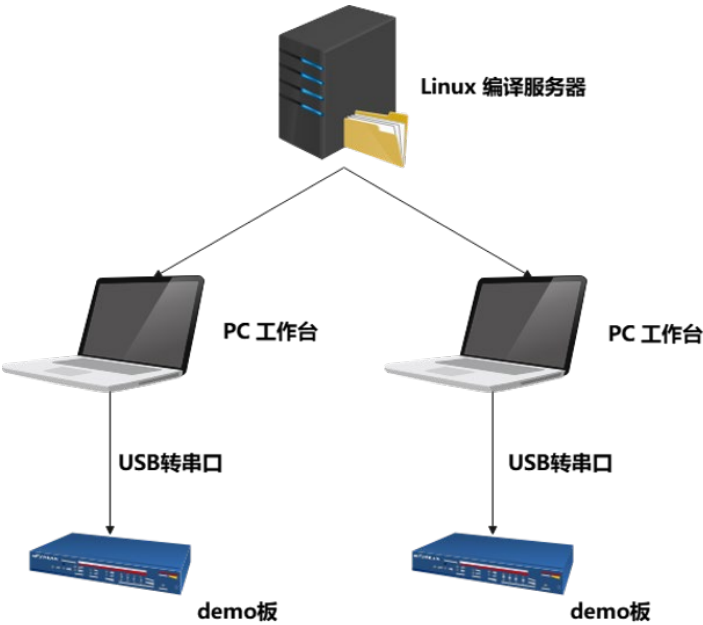
- PC 工作台

工作台主要用于目标板烧录和调试，通过串口与目标板连接，开发人员可以在工作台中烧录目标板的镜像、调试程序。工作台通常需要安装终端工具，用于登录 Linux 服务器和目标板，查看目标板的打印输出信息。工作台一般为 Windows 或 Linux 操作系统，在 Windows 或 Linux 工作台运行的终端工具通常有 SecureCRT、Putty、miniCom 等，这些软件需要从其官网下载。

- 目标板

本文的目标板以 demo 板为例，demo 板与工作台通过 USB 转串口连接。工作台将交叉编译出来的 demo 板镜像通过串口烧录到 demo 板，如图 1-1 所示。

图1-1 SDK 开发环境



1.2 Python 环境安装

- 步骤 1 打开“Linux 编译服务器”，输入命令“python3 -V”，查看 Python 版本号，推荐 Python3.8 以上版本。
- 步骤 2 如果 Python 版本太低，请使用命令“sudo apt-get update”更新系统到最新，或通过命令“sudo apt-get install python3 -y”安装 Python3（需 root/sudo 权限安装），安装后再次确认 Python 版本。
- 如果仍不能满足版本要求，请从“<https://www.python.org/downloads/source/>”下载对应版本源码包，下载与安装方法请阅读<https://wiki.python.org/moin/BeginnersGuide/Download> 和源码包内 README 内容。
- 步骤 3 安装 Python 包管理工具，运行命令“sudo apt-get install python3-setuptools python3-pip -y”（需 root/sudo 权限安装）。
- 步骤 4 参考表 1-1，安装依赖库，配置 WS73 的编译环境。

表1-1 Python 依赖库

依赖库	安装命令
Kconfiglib	pip3 install kconfiglib>=13.2.0

依赖库	安装命令
13.2.0+	
pycparser 2.21+	pip3 install pycparser>=2.21

---结束

2 SDK 环境搭建

- 2.1 SDK 目录结构介绍
- 2.2 编译 SDK

2.1 SDK 目录结构介绍

目录结构如下：

.		
—	application	# 用户态二进制文件和示例代码
—	build	# SDK构建所需配置文件和编译脚本
—	driver	# WS73的驱动文件
—	android_autoconfig.h	# Android配置文件
—	android.config	# Android配置文件
—	bsle	# WS73 BLE/SLE驱动文件
—	Makefile	# WS73 驱动编译入口
—	platform	# WS73 平台文件
—	wifi	# WS73 WiFi驱动文件
—	firmware	# WS73 驱动依赖固件
—	e	# 串口版本固件
—	us	# SDIO/USB版本固件
—	include	# API头文件存放目录
—	Kconfig	# Menuconfig配置总入口

└─ Makefile	# Makefile编译总入口
└─ open_source	# 存放开源组件及相关patch
└─ output	# 编译时生成的目标文件和中间文件

2.2 编译 SDK

2.2.1 config 配置方法

首次运行

用户首次运行时，根目录下执行 “vi build/config/ws73_default.config” 命令，配置相关参数：

编译参数

WS73 编译相关参数如表 2-1 所示。

表2-1 编译参数配置说明

参数	说明	示例
WSCFG_USING_GCC	是否配置编译器类型为 GCC	WSCFG_USING_GCC=y
WSCFG_CROSS_COMPILE	配置编译器存放路径	WSCFG_CROSS_COMPILE=/opt/toolchains/bin/arm-himix100-linux-
WSCFG_KERNEL_DIR	配置内核存放路径	WSCFG_KERNEL_DIR=/home/share/kernel/linux-4.9.y
WSCFG_ARCH_ARM	是否配置 CPU 架构类型为 ARM	WSCFG_ARCH_ARM=y
WSCFG_BUS_USB	是否配置通信总线类型为 USB	WSCFG_BUS_USB=y

图2-1 编译参数配置示例

```
#
# Configure the compilation environment
#
WSCFG_USING_GCC=y
# WSCFG_USING_LLVM_CLANG is not set
WSCFG_CROSS_COMPILE="/opt/toolchains/xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx/bin/arm-himix100-linux-"
WSCFG_CLANG_EXTRA_FLAGS=""
WSCFG_KERNEL_DIR="/home/share/xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx/kernel/linux-4.9.y"
WSCFG_ARCH_ARM=y
# WSCFG_ARCH_CUSTOM is not set
WSCFG_ARCH_NAME="arm"
BOARD_ASIC=y
BOARD_ASIC_WIFI=y
# WSCFG_BUS_SDIO
WSCFG_BUS_USB=y
# WSCFG_BUS_UART is not set
WSCFG_LINUX=y
PRE_OS_VERSION_LINUX=0
PRE_OS_VERSION_WIN32=1
PRE_OS_VERSION_WINDOWS=2
PRE_OS_VERSION_RAW=3
PRE_OS_VERSION_HIRTOS=4
PRE_OS_VERSION_WIN32_RAW=5
PRE_OS_VERSION_LITEOS=6
PRE_OS_VERSION_ANDROID=7
PRE_OS_VERSION=0
# end of build
```

说明

- WSCFG_CROSS_COMPILE 和 WSCFG_KERNEL_DIR 需读者依照实际情况进行配置。
- 若非 ARM 架构，可关闭 WSCFG_ARCH_ARM，打开 WSCFG_ARCH_CUSTOM 宏，并修改 WSCFG_ARCH_NAME，如 WSCFG_ARCH_NAME="mips"。
- 若非 SDIO 总线，可关闭 WSCFG_BUS_SDIO，打开对应总线宏，如 WSCFG_BUS_USB。

平台参数

WS73 平台相关参数如表 2-2 所示。

表2-2 平台参数配置说明

参数	说明	示例
CONFIG_FIRMWARE_BIN_PATH	配置 ws73.bin 文件路径	CONFIG_FIRMWARE_BIN_PATH="/etc/ws73/ws73.bin"
CONFIG_FIRMWARE_WIFICALI_PATH	配置 wifi_cali.bin 文件路径	CONFIG_FIRMWARE_WIFICALI_PATH="/etc/ws73/wifi_cali.bin"
CONFIG_FIRMWARE_BSLECALI_PATH	配置 btc_cali.bin 文件路径	CONFIG_FIRMWARE_BSLECALI_PATH="/etc/ws73/btc_cali.bin"
CONFIG_FIRMWARE_WOW_PATH	配置 wow.bin 文件路径	CONFIG_FIRMWARE_WOW_PATH="/etc/ws73/wow.bin"
CONFIG_INI_FILE_PATH	配置 ws73_cfg.ini 文件路径	CONFIG_INI_FILE_PATH="/etc/ws73_cfg.ini"

参数	说明	示例
CONFIG_PLAT_DFR_OUTPUT_PATH	配置 ws73 panic 文件存在路径	CONFIG_PLAT_DFR_OUTPUT_PATH="/etc/ws73"

图2-2 平台参数配置示例

```
#
# Platform
#
_PRE_PLAT_SHA256SUM_CHECK=y
WSCFG_ONEIMAGE=y
WSCFG_PLAT_VARIABLE_FEATTRUE=y
WSCFG_PLAT_DIAG_LOG_OUT=y
# CONFIG_DIAG_SUPPORT_SOCKET is not set
CONFIG_DIAG_SUPPORT_UART=y
CONFIG_DIAG_SUPPORT_LOCAL_LOG=y
CONFIG_PLAT_SUPPORT_DFR=y
CONFIG_PLAT_DFR_OUTPUT_PATH="/etc/ws73"
CONFIG_HCC_SUPPORT_TEST=y
CONFIG_HCC_ERROR_PRINT=y
_PRE_PLAT_HCC_SDI0=y
BT_EM_BUFFER_CALI_SUPPORT=y
# _PRE_PLAT_SLP_UART_FORWARD is not set
CONFIG_DFX_SUPPORT_SYSFS=y
# CONFIG_SDI0_RESCAN is not set

#
# Configure the loading path of files such as firmware
#
CONFIG_FIRMWARE_BIN_PATH="/etc/ws73/ws73.bin"
CONFIG_FIRMWARE_WIFICALI_PATH="/etc/ws73/wifi_cali.bin"
CONFIG_FIRMWARE_BSLECALI_PATH="/etc/ws73/btc_cali.bin"
CONFIG_FIRMWARE_WOW_PATH="/etc/ws73/wow.bin"
CONFIG_INI_FILE_PATH="/etc/ws73_cfg.ini"
# end of Configure the loading path of files such as firmware
```

说明

- 配置 CONFIG_FIRMWARE_BIN_PATH、CONFIG_FIRMWARE_WIFICALI_PATH、CONFIG_FIRMWARE_BSLECALI_PATH、CONFIG_FIRMWARE_WOW_PATH等参数后，在加载 ko 前，需要保证单板对应目录下，有对应的二进制文件。
- 配置 CONFIG_INI_FILE_PATH 后，在加载 ko 前，需要保证单板对应目录下，有对应的 ini 文件。
- 配置 CONFIG_PLAT_DFR_OUTPUT_PATH 需要保证该路径随时可写，以便出现系统 panic 时，能正常保存日志

重复运行

用户执行 “make” 完成编译后，如需重新配置驱动参数，可以在根目录下执行 “vi build/config/ws73_default.config” 进行配置。

2.2.2 驱动代码编译

根目录下执行 “make” 指令运行脚本编译，即可编译出对应的 SDK 程序。编译命令列如表 2-3 所示。

表2-3 make 参数列表

参数	示例	说明
无	make	默认执行全量编译。
all	make all	执行全量编译。
工程名称	make demo	参数输入 app 工程目录名称，启动增量编译，编译 app 工程是所输入的工程目录名称。默认是 demo 工程。
clean	make clean	清理编译过程中生成的中间文件和烧写文件。
hso	make hso	生成 DebugKit 应用数据库，用于驱动维测功能

2.2.3 注意事项

- 编译过程中，报错找不到某个包，请检查环境中的 Python 版本是否满足要求，相关组件是否已安装。
- 参见《WS73V100 常见问题 FAQ》中 “SDK 开发环境和 SDK 使用类” 章节。

3 Linux 移植说明

说明

WS73 支持 USB、SDIO、USB/URAT 三种总线模式，用户移植前，先确认目标开发板的 WS73 模组使用的总线类型。（不同总线类型差异可参考《NW117V100 系列 Wi-Fi、BLE 和 SLE Combo 芯片 用户指南》1.1 概述）

- [3.1 内核参数配置](#)
- [3.2 单板配置文件配置](#)
- [3.3 驱动加载步骤](#)
- [3.4 Linux 平台 WiFi 移植](#)
- [3.5 Linux 平台蓝牙移植](#)
- [3.6 Linux 平台星闪移植](#)

3.1 内核参数配置

3.1.1 配置 CFG80211

CFG80211 也是内核中 Wi-Fi 驱动和用户态进程的标准接口。

步骤 1 进入内核源码目录，执行 make menuconfig 命令，进入 Networking support → Wireless，设置 cfg80211 和 mac80211。

----结束

图3-1 配置 CFG80211

```
--- Wireless
[*>] cfg80211 - wireless configuration API
[ ]   nl80211 testmode command
[ ]   enable developer warnings
[*]   enable powersave by default
[ ]   cfg80211 wireless extensions compatibility
[ ]   lib80211 debugging messages
<M>   Generic IEEE 802.11 Networking Stack (mac80211)
      Default rate control algorithm (Minstrel) --->
[ ]   Enable mac80211 mesh networking (pre-802.11s) support
[ ]   Trace all mac80211 debug messages
[ ]   Select mac80211 debugging features ----
```

说明

如果内核中的 CFG80211 设置为'M', cfg80211 会编译成 cfg80211.ko 文件; 如果 CFG80211 设置为 '*', cfg80211 会被集成到内核中。

3.1.2 配置 SDIO

当使用 SDIO 模组设备时, 需要开启 MMC/SD/SDIO card support。

步骤 1 进入内核源码目录, 执行 make menuconfig 命令, 进入 Device Drivers → MMC/SD/SDIO card support, 按图 3-2 进行配置。

----结束

图3-2 配置 SDIO

```
-- MMC/SD/SDIO card support
[ ] MMC debugging
<*> HW reset support for eMMC
<*> Simple HW reset support for MMC
*** MMC/SD/SDIO Card Drivers ***
<*> MMC block device driver
(8)   Number of minors per block device
[*]   Use bounce buffer for simple hosts
< > SDIO UART/GPS class support
< > MMC host test driver
*** MMC/SD/SDIO Host Controller Drivers ***
<*> Secure Digital Host Controller Interface support
<*> SDHCI platform and OF driver helper
< > SDHCI OF support for the Atmel SDMMC controller
< > SDHCI support for Fujitsu Semiconductor F_SDH30
< > Winbond W83L51xD SD/MMC Card Interface support (NEW)
< > MMC/SD/SDIO over SPI
< > VUB300 USB to SDIO/SD/MMC Host Controller support
< > USB SD Host Controller (USHC) support
< > Renesas USDHI6R0L0 SD/SDIO Host Controller support
< > MediaTek SD/MMC Card Interface support
< > Command Queue Support
```

3.1.3 配置 USB

当使用 USB 模组设备时，需要开启 USB Support。

步骤 1 进入内核源码目录，执行 make menuconfig 命令，进入 Device Drivers → USB support，设置 “xHCI HCD (USB 2.0) support” 为 “*”。

----结束

图3-3 配置 USB

```

--- USB support
<*> Support for Host-side USB
[ ] USB announce new devices
    *** Miscellaneous USB options ***
[*] Enable USB persist by default
[ ] Dynamic USB minor allocation
[ ] OTG support
[ ] Rely on OTG and EH Targeted Peripherals List
< > USB Monitor
< > Support WUSB Cable Based Association (CBA)
    *** USB Host Controller Drivers ***
< > Cypress C67x00 HCD support
<*> xHCI HCD (USB 3.0) support
- *- Generic xHCI driver for a platform device
<*> EHCI HCD (USB 2.0) support
[*] Root Hub Transaction Translators
[*] Improved Transaction Translator scheduling
<*> Generic EHCI driver for a platform device

```

3.1.4 配置 Netlink

由于 wpa_supplicant、hostapd 应用采用 Netlink 技术与 Linux 内核通信，需要配置 Netlink。

步骤 1 进入内核源码目录，执行 make menuconfig 命令。选择 Networking support → Networking options，设置 “Network packet filtering framework (Netfilter)” 为 “y”（选择为 “*”）。

步骤 2 进入 Network packet filtering framework (Netfilter)，设置 “Advanced netfilter Configuration” 为 “y”（选择为 “*”）。

----结束

图3-4 配置 Netlink

```

--- Network packet filtering framework (Netfilter)
[ ] Network packet filtering debugging (NEW)
[*] Advanced netfilter configuration (NEW)
<M> Bridged IP/ARP packets filtering (NEW)
Core Netfilter Configuration --->
< > IP set support (NEW) ----
< > IP virtual server support (NEW) ----
IP: Netfilter Configuration --->
IPv6: Netfilter Configuration --->

```

3.1.5 配置 NAT 转发（可选）

如果需要使用 SoftAP 网络共享功能，需要配置 NAT 转发功能。

- 步骤 1 进入内核源码目录，执行 make menuconfig 命令。选择 Networking support → Networking options → Network packet filtering framework (Netfilter) → Core Netfilter Configuration，设置 “Netfilter connection tracking support” 为 “y”（选择为 “*”）。
- 步骤 2 再进入 Networking support → Networking options → Network packet filtering framework (Netfilter)，根据使用需求配置 NAT 转发功能。

----结束

图3-5 配置 NAT 转发

```
<*> Netfilter IPv4 packet duplication to alternate destination
<*> ARP packet logging
<*> IPv4 packet logging
<*> IPv4 packet rejection
<*> IP tables support (required for filtering/masq/NAT)
< > "ah" match support
< > "ecn" match support (NEW)
< > "ttl" match support (NEW)
< > Packet filtering (NEW)
< > Packet mangling (NEW)
< > raw table support (required for NOTRACK/TRACE) (NEW)
<*> ARP tables support
< > ARP packet filtering (NEW)
< > ARP payload mangling (NEW)
```

3.2 单板配置文件配置

具体内容请参见《WS73V100 单板配置文件说明书》中“GPIO 配置”小节。

3.3 驱动加载步骤

- 步骤 1 默认加载 cfg80211.ko
- 步骤 2 确认采用的驱动加载模式，若通过配置启动脚本来加载驱动，可直接跳到步骤 4；若通过适配驱动模组设备号，再加载驱动，可继续执行步骤 3。

步骤 3 针对 USB 模组，可执行 “lsusb” 查看模组的 VID 和 PID 号，其中 WS73 模组默认值为 ffff:3733。

```
/komod # lsusb
Bus 001 Device 001: ID 1d6b:0002
Bus 001 Device 002: ID ffff:3733
Bus 002 Device 001: ID 1d6b:0003
```

针对 SDIO 模组，可查看内核实际注册的设备节点来获取模组的 VID 和 PID 号，其中 WS73 模组默认值为 ffff:3733。

```
/ # cat /sys/bus/sdio/devices/mmcl\:\0001\:1/uevent
SDIO_CLASS=07
SDIO_ID=FFFF:3733
MODALIAS=sdio:c07vFFFFd3733
/ #
```

步骤 4 先加载 plat_soc.ko，再根据需要加载 wifi_soc.ko、sle_soc.ko、ble_soc.ko（wifi、sle、ble 间无依赖关系，只要在 plat ko 后加载即可）

----结束

3.4 Linux 平台 WiFi 移植

3.4.1 驱动移植步骤

步骤 1 解压驱动源码包。

将驱动源码包 WS73V100R001C00SPCXXX.tar.gz 放置于服务器上，并进行解压，参考 2.2 编译 SDK 配置编译工具链、内核路径、CPU 架构等，命令如下：

```
$ tar -xzf WS73V100R001C00SPCXXX.tar.gz
$ cd WS73V100R001C00SPCXXX
```

步骤 2 完成配置后，执行 make all 命令编译驱动文件，在 output 生成目标文件：

```
$ make all
```

编译结果输出到"output"目录下，如表 3-1 所示。

表3-1 全量编译结果

文件名	说明
plat_soc.ko	WS73 平台驱动模块。
wifi_soc.ko	WS73WiFi 驱动模块。

文件名	说明
ble_soc.ko	WS73 蓝牙驱动模块。
sle_soc.ko	WS73 星闪驱动模块。
ws73_cfg.ini	WS73 客制化的配置文件。

----结束

📖 说明

- 编译工具链接需要提前配置环境变量，例如 `export PATH=$PATH:/home/wifi/share/arm-linux-gnueabi/bin`，路径根据实际情况配置。
- 使用不同工具链时，可能会导致编译过程中出现警告导致编译失败，可在相关驱动组件的 Makefile 文件中修改编译选项。

3.4.2 工具移植步骤

步骤 1 用户选择所需的第三方组件进行下载，相关组件下载链接如下：

- libnl-3.5.0 下载链接：
<http://www.linuxfromscratch.org/blfs/view/svn/basicnet/libnl.html>
- wpa_supplicant-2.10、hostapd-2.10 下载链接：<http://w1.fi/releases>
- openssl-1.1.1n 下载链接：<https://www.openssl.org/source/old/1.1.1>

步骤 2 将下载压缩包放置 SDK 相应目录，并执行下列命令：

- 解压 libnl-3.5.0 至 SDK 的 `open_source/libnl` 目录下。
- 解压 openssl-1.1.1 至 SDK 的 `open_source/openssl` 目录下。
- 解压 wpa_supplicant-2.10 至 SDK 的 `open_source/wpa_supplicant` 目录下。
- 解压 hostapd-2.10 至 SDK 的 `open_source/hostapd` 目录下。

```
$ mkdir -p open_source/libnl && tar -xzf libnl-3.5.0.tar.gz -C open_source/libnl/ --strip-component=1
$ mkdir -p open_source/openssl && tar -xzf openssl-1.1.1n.tar.gz -C open_source/openssl/ --strip-component=1
$ mkdir -p open_source/wpa_supplicant && tar -xzf wpa_supplicant-2.10.tar.gz -C open_source/wpa_supplicant/ --strip-component=1
$ mkdir -p open_source/hostapd && tar -xzf hostapd-2.10.tar.gz -C open_source/hostapd/ --strip-component=1
```

步骤 3 给 wpa_supplicant 组件补丁，相关命令如下：

```
$ cd open_source/wpa_supplicant (以用户实际路径为准)
$ patch -p1 < wpa_supplicant_2_10_linux.patch
```

步骤 4 完成对相关组件的补丁后，在 SDK 根目录下执行“make tools”命令，在 output 目录下生成目标文件如下：

```
wifi_service/libnl-3.so.200.26.0
wifi_service/libnl-genl-3.so.200.26.0
wifi_service/libnl-route-3.so.200
wifi_service/libcrypto.so.1.1
wifi_service/libssl.so.1.1
wifi_service/wpa_supplicant
wifi_service/wpa_cli
wifi_service/hostapd
```

----结束

说明

提供的 wpa_supplicant.patch 是基于 wpa_supplicant 2.10 生成的补丁文件，若用户使用其他版本的开源组件，可能会产生兼容性问题。

3.4.3 WiFi 业务调试

参考《WS73V100 Linux 软件开发指南》。

3.5 Linux 平台蓝牙移植

3.5.1 驱动移植步骤

步骤 1 解压驱动源码包。

将驱动源码包 WS73V100R001C00SPCXXX.tar.gz 放置于服务器上，并进行解压，参考 2.2 编译 SDK 配置编译工具链、内核路径、CPU 架构等，命令如下：

```
$ tar -xzf WS73V100R001C00SPCXXX.tar.gz
$ cd WS73V100R001C00SPCXXX
```

步骤 2 完成配置后，执行 make all 命令编译驱动文件，在 output 生成目标文件：

```
$ make all
```

----结束

说明

- 编译工具链接需要提前配置环境变量，例如 `export PATH=$PATH:/home/AIoT/share/arm-himix100-linux/bin`，路径根据实际情况配置。
- 使用不同工具链时，可能会导致编译过程中出现告警导致编译失败，可在 Makefile 文件中修改编译选项。

3.5.2 工具移植步骤

步骤 1 下载第三方工具 bluez-5.64.tar.xz 及其依赖库，需要下载对应版本的源码压缩文件，下载链接如下：

```
bluez-5.64.tar.xz--https://mirrors.edge.kernel.org/pub/linux/bluetooth/bluez-5.64.tar.xz
dbus-1.12.20.tar.gz--https://dbus.freedesktop.org/releases/dbus/dbus-1.2.20.tar.gz
expat-2.4.6.tar.gz--
https://github.com/libexpat/libexpat/releases/download/R_2_4_6/expat-2.4.6.tar.gz
gettext-0.21.tar.gz--https://ftp.gnu.org/gnu/gettext/gettext-0.21.tar.gz
json-c-0.13.tar.gz--http://sources.buildroot.net/json-c/json-c-0.13.tar.gz
libffi-3.3.tar.gz--https://gcc.gnu.org/pub/libffi/libffi-3.3.tar.gz
libical-1.0.tar.gz--https://src.fedoraproject.org/repo/pkgs/libical/libical-1.0.tar.gz
ncurses-6.3.tar.gz--https://ftp.gnu.org/gnu/ncurses/ncurses-6.3.tar.gz
pcre-8.45.tar.gz--https://ftp.exim.org/pub/pcre/pcre-8.45.tar.gz
readline-8.1.tar.gz--https://ftp.gnu.org/gnu/readline/readline-8.1.tar.gz
zlib-1.2.11.tar.gz--https://src.fedoraproject.org/repo/pkgs/R/zlib-1.2.11.tar.gz
glib-2.40.0.tar.xz--https://src.fedoraproject.org/repo/pkgs/glib2/glib-2.40.0.tar.xz
```

步骤 2 下载压缩文件并放至 SDK 相应目录下。

- 解压 bluez-5.64.tar.xz 至 SDK 的 open_source 目录下。

```
tar -xvf bluez-5.64.tar.xz
```

- 解压依赖库至 SDK 的 open_source 目录下。

```
tar zxvf expat-2.4.6.tar.gz
tar zxvf libical-1.0.tar.gz
tar zxvf dbus-1.12.20.tar.gz
tar zxvf zlib-1.2.11.tar.gz
tar zxvf libffi-3.3.tar.gz
tar zxvf ncurses-6.3.tar.gz
tar zxvf readline-8.1.tar.gz
```

```
tar zxvf pcre-8.45.tar.gz
tar zxvf gettext-0.21.tar.gz
tar -xvf glib-2.40.0.tar.xz
```

步骤 3 添加对 glib 的 patch 修改，步骤如下：

- 在 glib 目录下新建 glib.patch 文件，内容如下：

```
--- ./glib-2.40.0/glib/gdate.c 2014-02-12 20:08:36.000000000 +0000
+++ gdate.c 2023-04-06 07:06:39.658647456 +0000
@@ -2439,6 +2439,9 @@
 *
 * Returns: number of characters written to the buffer, or 0 the buffer was too small
 */
+#pragma GCC diagnostic push
+
+#pragma GCC diagnostic ignored "-Wformat-nonliteral"
gsize
g_date_strftime (gchar      *s,
                 gsize      slen,
@@ -2549,3 +2552,4 @@
    return retval;
#endif
}
+#pragma GCC diagnostic pop
\ No newline at end of file
```

- 在 glib 目录执行如下指令

```
patch -p1 < ./glib.patch
```

步骤 4 添加对 bluez 的 patch 修改，相关步骤如下：

- 在 bluez 源码目录下新建 bluez.patch 文件，内容如下：

```
--- ./bluez-5.64/tools/test-runner.c 2022-03-16 15:06:20.000000000 +0000
+++ test-runner.c 2023-04-06 09:26:23.566585603 +0000
@@ -35,6 +35,10 @@
#include "lib/hci_lib.h"
#include "tools/hciattach.h"
+#ifndef MS_STRICTATIME
+#define MS_STRICTATIME (1 << 24)
+#endif
+
+#ifndef WAIT_ANY
#define WAIT_ANY (-1)
#endif
```


- 在 bluez 目录执行如下指令

```
patch -p1 < ./bluez.patch
```

步骤 5 编译 bluez 依赖库，相关指令如下：

- 配置环境变量

```
export PKG_CONFIG_LIBDIR=/vendor/lib/pkgconfig/
```

- 编译 expat

```
cd expat-2.4.6/
```

```
./configure --prefix=/vendor --host=arm-himix100-linux CC=arm-himix100-linux-gcc  
CFLAGS="-fPIC -fstack-protector-all" LDFLAGS="-Wl,-z,now -Wl,-z,relro -Wl,-z,noexecstack"  
make  
make install
```

- 编译 libical

```
cd libical-1.0/
```

```
export CC=arm-himix100-linux-gcc  
cmake -DCMAKE_INSTALL_PREFIX=/vendor  
make -j8  
make install
```

- 编译 dbus

```
cd dbus-1.12.20/
```

```
./configure --prefix=/vendor --host=arm-himix100-linux --with-x=no --enable-abstract-sockets  
CC=arm-himix100-linux-gcc CFLAGS="-I/vendor/include -fstack-protector-all -fPIC -fPIE -pie"  
LDFLAGS="-L/vendor/lib -Wl,-z,now -Wl,-z,relro -Wl,-z,noexecstack -fstack-protector-all"  
make  
make install
```

- 编译 zlib

```
cd zlib-1.2.11/
```

```
CC=arm-himix100-linux-gcc prefix=/vendor CFLAGS="-fstack-protector-all -fPIC"  
LDFLAGS="-L/vendor/lib -Wl,-z,now -Wl,-z,relro -Wl,-z,noexecstack" ./configure  
make  
make install
```

- 编译 libffi

```
cd libffi-3.3
```

```
./configure --prefix=/vendor CC=arm-himix100-linux-gcc --host=arm-himix100-linux  
CFLAGS="-fPIC -fstack-protector-all" LDFLAGS="-Wl,-z,now -Wl,-z,relro -Wl,-z,noexecstack"  
make  
make install
```

- 编译 ncurses

```
cd ncurses-6.3
./configure --disable-stripping --host=arm-himix100-linux --prefix=/vendor CC=arm-himix100-
linux-gcc CFLAGS="-fPIC -fstack-protector-all" LDFLAGS="-Wl,-z,now -Wl,-z,relro -Wl,-
z,noexecstack"
make
make install
```

- 编译 readline

```
cd readline-8.1
./configure --prefix=/vendor --host=arm-himix100-linux CC=arm-himix100-linux-gcc --enable-
shared --enable-static bash_cv_wcwidth_broken=yes CFLAGS="-I/vendor/include -fstack-
protector-all -fPIC " LDFLAGS="-L/vendor/lib -Wl,-z,now -Wl,-z,relro -Wl,-z,noexecstack"
make -j4 SHLIB_LIBS=-lncurses
make install
```

- 编译 pcre

```
cd pcre-8.45
./configure --prefix=/vendor --host=arm-himix100-linux CC=arm-himix100-linux-gcc CXX=arm-
himix100-linux-g++ CFLAGS="-I/vendor/include -fstack-protector-all -fPIC" LDFLAGS="-
L/vendor/lib -Wl,-z,now -Wl,-z,relro -Wl,-z,noexecstack"
make
make install
```

- 编译 gettext

```
cd gettext-0.21
./configure --prefix=/vendor --host=arm-himix100-linux CC=arm-himix100-linux-gcc CXX=arm-
himix100-linux-g++ CFLAGS="-fPIC -fstack-protector-all" LDFLAGS="-L/vendor/lib -Wl,-z,now
-Wl,-z,relro -Wl,-z,noexecstack"
make -j4
make install
```

- 编译 glib

```
cd glib-2.40.0
./configure --prefix=/vendor --host=arm-himix100-linux CC=arm-himix100-linux-gcc
CFLAGS="-I/vendor/include -fPIC -fstack-protector-all" CPPFLAGS=-I/vendor/include
LDFLAGS="-L/vendor/lib -Wl,-z,now -Wl,-z,relro -Wl,-z,noexecstack" glib_cv_stack_grows=no
glib_cv_uscore=yes ac_cv_func_posix_getpwuid_r=yes ac_cv_func_posix_getgrgid_r=yes
make
make install
```

步骤 6 编译 bluez 协议栈及用户态工具，相关指令如下：

- 配置环境变量

```
export PKG_CONFIG_LIBDIR=$PKG_CONFIG_LIBDIR:/usr/lib/x86_64-linux-
gnu/pkgconfig:/usr/share/pkgconfig/
```

- 编译 bluez

```
cd bluez-5.64
autoreconf
cp ../bluez_configure/configure .
./configure --prefix=/vendor --host=arm-himix100-linux CC=arm-himix100-linux-gcc --enable-
deprecated --enable-tools --enable-cups --enable-test CFLAGS="-I/vendor/include -fstack-
protector-all -fPIC -fPIE -pie" LDFLAGS="-fstack-protector-all -L/vendor/lib -Wl,-z,now -Wl,-
z,relro -Wl,-z,noexecstack -lpcre" --enable-experimental -enable-maintainer-mode --disable-
udev --enable-pie --enable-shared --enable-static &&sed -i 's/^MISC_LDFLAGS = -pie -Wl,-
z,now/MISC_LDFLAGS = -pie -Wl,-z,now/g' Makefile&& sed -i s/-Werror//g ./Makefile
make -j4
make install
```

- 交叉编译环境没有 kernel 头文件，make 会报如下的错误

```
linux/uhid.h:No such file or directory
```

可将系统 kernel 头文件拷贝到 vendor 目录供编译引用，执行如下命令，重新编译

```
cp -r /usr/include/linux /vendor/include
```

步骤 7 生成目标文件在 vendor 目录，如表 3-2 所示。

表3-2 生成目标文件目录

目录	说明
/vendor/lib	依赖库动态库文件： libglib-2.0.so.0 libexpat.so.1 libpcre.so.1 libdbus-1.so.3 libintl.so.8 libreadline.so.8
/vendor/bin	dbus 及 bluez 相关工具可执行文件： bluetoothctl dbus-daemon hciconfig hcidtool
/vendor/share/d bus-1	dbus 配置文件： session.conf system.conf

目录	说明
/vendor/libexec/ bluetooth	bluez 协议栈可执行文件： bluetoothd

----结束

 说明

- 编译工具链接需要提前配置环境变量，例如 export PATH=\$PATH:/home/AIoT/share/arm-himix100-linux/bin，路径根据实际情况配置。
- --prefix=/vendor 目录是对应工具库所安装的位置，dbus 运行依赖/vendor/share/dbus-1 该位置下的配置文件，可根据系统版本情况配置。

3.5.3 蓝牙业务调试

参考《WS73V100 Linux 软件开发指南》。

3.6 Linux 平台星闪移植

3.6.1 驱动移植步骤

步骤 1 解压驱动源码包。

将驱动源码包 WS73V100R001C00SPCXXX.tar.gz 放置于服务器上，并进行解压，参考 2.2 编译 SDK 配置编译工具链、内核路径、CPU 架构等，命令如下：

```
$ tar -xzf WS73V100XXX.tar.gz
$ cd WS73V100XXX
```

步骤 2 完成配置后，执行 make all 命令编译驱动文件，在 output 生成目标文件：

```
$ make all
```

----结束

 说明

- 编译工具链接需要提前配置环境变量，例如 export PATH=\$PATH:/home/AIoT/share/arm-himix100-linux/bin，路径根据实际情况配置。

- 使用不同工具链时，可能会导致编译过程中出现警告导致编译失败，可在 Makefile 文件中修改编译选项。

3.6.2 星闪工具获取

步骤 1 进入 sdk 的 application/bin 目录。

步骤 2 根据 host 版本选择对应版本。

----结束

3.6.3 星闪业务调试

3.6.3.1 星闪设备检测

SDIO 星闪设备检测

SDIO 星闪在使用前 Host 端需要先检测到 SDIO 设备：

- 如果星闪芯片是默认上电的，则 Linux 启动时会检测到 SDIO 设备，显示 “mmc1: new SDIO card at address 0001” 打印信息，说明 SDIO 检测成功。
- 如果星闪芯片是默认不上电的，则需要在星闪驱动中添加上电 SDIO 检测相关流程，再继续执行。

平台可以通过执行 “cat /proc/mci/mci_info” 命令查看是否检测到 SDIO 设备，如图 3-6 所示。

图3-6 SDIO 设备示例

```
~ # cat /proc/mci/mci_info
MCI0: unplugged_disconnected
MCI1: plugged_connected
      Type: SDIO card Mode: HS
      Speed Class: Class 0
      Uhs Speed Grade: Less than 10MB/sec(0h)
      Host work clock: 25MHz
      Card support clock: 25MHz
      Card work clock: 25MHz
      Card error count: 0
MCI2: invalid
.. ..
```

USB 星闪设备检测

USB 星闪在使用前 Host 端需要先检测到 USB 设备：

- 如果主控平台支持 USB 功能且是 build-in 模式，则 Linux 启动时会检测到 USB 设备，显示 “xHCI Host Controller” 打印信息，说明探测到 USB 设备。
- 如果主控平台支持 USB 功能且需要加载驱动，则需要加载 usb 驱动后，再加载 WS73 驱动。

平台可以通过执行 “lsusb” 命令查看是否检测到 USB 设备，如图 3-7 所示。

图3-7 USB 设备示例

```
~ # lsusb
Bus 001 Device 001: ID 1d6b:0002
Bus 001 Device 006: ID ffff:3733 ← USB SLE
Bus 002 Device 001: ID 1d6b:0003
~ "
```

3.6.3.2 加载驱动

步骤 1 将驱动依赖的 bin 文件和 ini 文件分别拷贝至/etc/ws73 目录和/etc 目录（Host 侧默认配置路径）。

步骤 2 将编译出的驱动拷贝至单板，依次加载 plat_ws73.ko, gle_soc.ko。

```
$ insmod plat_ws73.ko
$ insmod gle_soc.ko
```

步骤 3 串口打印如所示，则说明星闪驱动初始化成功。

图3-8 星闪驱动加载成功

```
[HCC] sle host init finished

----结束
```

3.6.3.3 载入工具

步骤 1 将星闪执行需要的工具 “build/sparklinkd” “build/sparklinkctrl” 复制到/bin 目录下；

步骤 2 在 bin 目录下修改工具的可执行权限：

```
chmod a+x sparklinkd
```

```
chmod a+x sparklinkctrl
```

----结束

3.6.3.4 星闪基础业务操作示例

步骤 1 加载驱动。

```
$ insmod plat_ws73.ko  
$ insmod sle_soc.ko
```

步骤 2 启动 sparklinkd，命令如下：

```
sparklinkd &
```

步骤 3 启动 sparklinkctrl，命令如下：

```
sparklinkctrl
```

说明

sparklinkctrl 命令执行成功会出现 “sparklinkctrl>” 符号。

步骤 4 注册广播回调函数

- 在'>'后执行 gle_dd_register_cbks

```
sparklinkctrl>gle_dd_register_cbks
```

图3-9 注册成功

```
gle_dd_register_cbks  
gle manager exter_cbks will be overlaped  
[BT_APP] gle adv register cbks = 0  
gle manager exter_cbks will be overlaped  
[BT_APP] gle scan register cbks = 0  
gle manager exter_cbks will be overlaped  
[BT_APP] gle basic register cbks = 0
```

步骤 5 设置广播参数

- 在'>'后执行 bth_gle_set_adv

```
sparklinkctrl>bth_gle_set_adv
```

图3-10 设置成功

```
[BT_APP] gle sample dd adv status cbk
[BT_APP] event = 0x02, adv_handle = 1, result = 0
mgr_adv_item->status = 1
update adv params success
gle hci command check send check and send, cmd_num:1
[HCI]gle hci internal evt cbk unreg table->id = 0xc04
```

步骤 6 开启广播

- 在'>'后执行 `bth_gle_enable_adv 1`, 开启广播

```
sparklinkctrl> bth_gle_enable_adv 1
```

图3-11 开启广播成功

```
bth_gle_enable_adv 1
[BT_APP] adv_handle = 1, adv_enable = 1
```

步骤 7 执行连接

- 在'>'后执行 `gle_at_cmd_connect 0 0 0x000000000000`

```
sparklinkctrl> gle_at_cmd_connect 0 0 0x000000000000
```

- 回显连接成功

```
[Connected]
```

步骤 8 卸载驱动

- 退出 `sparklinkd`、`sparklinkctrl` 等应用
- 卸载驱动:

```
$ rmmod gle_soc.ko
$ rmmod plat_ws73.ko
```

----结束

4 常见问题

4.1 驱动加载出错

4.2 wpa_supplicant 启动失败

4.1 驱动加载出错

问题描述

在加载驱动时，出现如下错误：

```
Unknown symbol cfg80211_inform_bss_frame_data (err 0)
Unknown symbol cfg80211_sched_scan_results (err 0)
Unknown symbol cfg80211_scan_done (err 0)
Unknown symbol cfg80211_sched_scan_stopped (err 0)
```

问题原因

加载 wifi_soc.ko 时，找不到 Linux 内核 CFG80211 相关的符号引用。

解决方案

- 1、如果 CFG80211 配置没有打开，可参考“3.1.1 配置 CFG80211”配置内核选项选项，并将重新编译后的内核烧录进目标开发板。
- 2、如果 CFG80211 配置为“M”，则需要将主控 Linux 内核目录下的 cfg80211.ko 拷贝至主控板，加载 cfg80211.ko 后，重新加载驱动。

4.2 wpa_supplicant 启动失败

问题描述

加载驱动完成后，启动 wpa_supplicant，但是启动失败。

解决办法

1. 为 wpa_supplicant 添加可执行权限。
2. 执行 wpa_supplicant 命令时，确定指定路径下存在 wpa_supplicant.conf 配置文件，且配置文件正确。
3. wpa_supplicant.conf 文件中会记录 wlan0 存放位置，确保该目录下具有读写权限。